

✓ GECS — FLANG-BERT Champion v10 (Task 1)

Model: SALT-NLP/FLANG-BERT | **Data:** cleaned_v10 | **Runtime:** Colab A100

Key change from v9c:

- No ModelInput column — text built in Dataset class
- SegmentDescription is the raw target text
- Sibling context constructed at training time
- Numeric features pulled directly from CSV columns
- API-friendly: same logic runs at inference

Training: 4 CE + 8 Focal (gamma=1.0) + 4 CE low LR | LLRD=0.95

```
import torch
if not torch.cuda.is_available():
    raise RuntimeError('No GPU. Runtime > Change runtime type > A100')
print(f'GPU : {torch.cuda.get_device_name(0)}')
print(f'VRAM : {torch.cuda.get_device_properties(0).total_memory/1e9:.1f} GB')
```

GPU : NVIDIA A100-SXM4-40GB
VRAM : 42.4 GB

```
!pip install transformers accelerate -q
print('Done.')
```

Done.

```
from google.colab import drive
drive.mount('/content/drive')
from pathlib import Path

BASE_DIR = Path('/content/drive/MyDrive/CAPSTONE')
CLEAN_DIR = BASE_DIR / 'cleaned_v10'
ART_DIR = BASE_DIR / 'flangbert_v10_artifacts'
ART_DIR.mkdir(parents=True, exist_ok=True)

FILE_T1 = CLEAN_DIR / 'task1_gecs_cleaned_v10.csv'
FILE_HIER = BASE_DIR / 'cleaned_v6' / 'industries_Hierarchy.csv'
SPLITS_FILE = CLEAN_DIR / 'canonical_splits.npz'

print('Path check:')
for f in [FILE_T1, FILE_HIER, SPLITS_FILE]:
    print((' OK' if f.exists() else ' NOT FOUND'), f.name)
```

Mounted at /content/drive
Path check:
OK task1_gecs_cleaned_v10.csv
OK industries_Hierarchy.csv
OK canonical_splits.npz

```
import time, json, warnings, pickle, copy
from datetime import datetime
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer, AutoModel, get_cosine_schedule_with_warmup
from sklearn.metrics import f1_score, accuracy_score
from sklearn.preprocessing import LabelEncoder, StandardScaler
warnings.filterwarnings('ignore')
device = torch.device('cuda')
print(f'Device: {device}')
```

Device: cuda

```
CFG = {
    'model_name' : 'SALT-NLP/FLANG-BERT',
```

```

'max_len'      : 512,      # covers p99 with sibling context
'dropout'      : 0.1,
'num_features' : 10,      # 7 original + 3 short flags
'use_bf16'     : True,

# Multi-task
'aux_weight_sector': 0.15,
'aux_weight_group' : 0.20,

# 3-phase: 4 CE + 8 Focal + 4 CE low LR
'phase1_epochs' : 6,
'phase2_epochs' : 4,
'phase3_epochs' : 4,
'lr_phase1'     : 2e-5,
'lr_phase2'     : 2e-5,
'lr_phase3'     : 5e-6,
'warmup_ratio'  : 0.1,
'lr_factor'     : 0.95,

# Batch
'batch_size_train' : 32,
'batch_size_eval'  : 64,

# Loss
'label_smoothing'  : 0.05,
'focal_gamma'      : 1.0,
'focal_weight_cap' : 2.0,
'weight_power'     : 0.5,

# Sibling context
'sibling_words'    : 25,      # max words per sibling description
'lp_words'         : 100,     # max words from LongProfile
'sector_group_dropout' : 0.25, # fraction of steps sector/group tokens are masked

'seed'             : 42,
}
torch.manual_seed(CFG['seed'])
np.random.seed(CFG['seed'])
print('Config - FLANG-BERT v10 | clean architecture | sibling context in Dataset')
print(f' Phases: {CFG["phase1_epochs"]} CE + {CFG["phase2_epochs"]} Focal + {CFG["phase3_epochs"]} CE-lowLR')

```

Config - FLANG-BERT v10 | clean architecture | sibling context in Dataset
 Phases: 6 CE + 4 Focal + 4 CE-lowLR

```

t1 = pd.read_csv(FILE_T1, dtype={'MstarGlobal': str})
hier = pd.read_csv(FILE_HIER, dtype={'industry_id': str})

splits = np.load(SPLITS_FILE)
t1_train_idx = splits['t1_train_idx']
t1_test_idx = splits['t1_test_idx']

le = LabelEncoder()
le.fit(t1['MstarGlobal'])
t1['label'] = le.transform(t1['MstarGlobal'])
NUM_CLASSES = len(le.classes_)

t1['sector_str'] = t1['MstarGlobal'].str[:3]
le_sec = LabelEncoder()
le_sec.fit(t1['sector_str'])
t1['sector_label'] = le_sec.transform(t1['sector_str'])
NUM_SECTORS = len(le_sec.classes_)

t1['group_str'] = t1['MstarGlobal'].str[:5]
le_grp = LabelEncoder()
le_grp.fit(t1['group_str'])
t1['group_label'] = le_grp.transform(t1['group_str'])
NUM_GROUPS = len(le_grp.classes_)

industry_to_sector = dict(zip(hier['industry_id'], hier['sector_name']))
t1_top_k_str = t1['MstarGlobal'].value_counts().head(10).index.tolist()
t1_top_k_enc = le.transform(t1_top_k_str).tolist()

print(f'Task 1 : {t1.shape}')
print(f'Train : {len(t1_train_idx):,} Test: {len(t1_test_idx):,}')
print(f'Classes: {NUM_CLASSES} leaf | {NUM_SECTORS} sector | {NUM_GROUPS} group')
print(f'Sample SD: {t1["SegmentDescription"].iloc[0][:150]}')

```

Task 1 : (52380, 30)
 Train : 41,657 Test: 10,723
 Classes: 145 leaf | 11 sector | 55 group
 Sample SD: frozen and vegetables segment includes the green giant and le sueur brands.

```
# — Numeric Features —————
NUMERIC_COLS = [
    'revenue_share', 'is_largest_bin', 'log_revenue',
    'log_total_revenue', 'n_segments', 'herfindahl_index',
    'report_quarter', 'lp_short_flag', 'sd_short_flag', 'sn_short_flag',
]
CONTINUOUS = ['revenue_share', 'log_revenue', 'log_total_revenue',
              'n_segments', 'herfindahl_index', 'report_quarter']

for col in NUMERIC_COLS:
    t1[col] = t1[col].fillna(0.0)

scaler = StandardScaler()
train_feats = t1.iloc[t1_train_idx][NUMERIC_COLS].copy()
test_feats = t1.iloc[t1_test_idx][NUMERIC_COLS].copy()
train_feats[CONTINUOUS] = scaler.fit_transform(train_feats[CONTINUOUS])
test_feats[CONTINUOUS] = scaler.transform(test_feats[CONTINUOUS])

train_numeric = train_feats[NUMERIC_COLS].values.astype(np.float32)
test_numeric = test_feats[NUMERIC_COLS].values.astype(np.float32)

with open(ART_DIR / 'numeric_scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)

print(f'Features ({len(NUMERIC_COLS)}): {NUMERIC_COLS}')
print(f'Train: {train_numeric.shape} Test: {test_numeric.shape}')
```

Features (10): ['revenue_share', 'is_largest_bin', 'log_revenue', 'log_total_revenue', 'n_segments', 'herfindahl_index', 'report_quarter', 'lp_short_flag', 'sd_short_flag', 'sn_short_flag']
 Train: (41657, 10) Test: (10723, 10)

```
# — Class Weights —————
def make_weights(labels, n_classes, power, cap=None):
    counts = np.bincount(labels, minlength=n_classes).astype(float)
    counts = np.where(counts == 0, 1, counts)
    w = (1.0 / counts) ** power
    w = w / w.mean()
    if cap is not None:
        w = np.clip(w, 0, cap)
        w = w / w.mean()
    return torch.tensor(w, dtype=torch.float32).to(device)

train_labels = t1.iloc[t1_train_idx]['label'].values
train_sec_labels = t1.iloc[t1_train_idx]['sector_label'].values
train_grp_labels = t1.iloc[t1_train_idx]['group_label'].values

class_weights_t = make_weights(train_labels, NUM_CLASSES, CFG['weight_power'], CFG['focal_weight_cap'])
sector_weights_t = make_weights(train_sec_labels, NUM_SECTORS, CFG['weight_power'])
group_weights_t = make_weights(train_grp_labels, NUM_GROUPS, CFG['weight_power'])

print(f'Leaf weights - min: {class_weights_t.min():.3f} max: {class_weights_t.max():.3f}')
print(f'Sector weights - min: {sector_weights_t.min():.3f} max: {sector_weights_t.max():.3f}')
```

Leaf weights - min: 0.293 max: 2.018
 Sector weights - min: 0.585 max: 1.507

```
# — Dataset - builds text input and pulls features directly -
import random as _random

tokenizer = AutoTokenizer.from_pretrained(CFG['model_name'])

class GECSDataset(Dataset):
    def __init__(self, df, indices, numeric_feats, tokenizer, max_len,
                 sibling_words=25, lp_words=100,
                 sector_group_dropout=0.25, is_train=True):
        self.df = df.reset_index(drop=False) # keep original index
        self.indices = indices
        self.numeric_feats = numeric_feats
        self.tokenizer = tokenizer
        self.max_len = max_len
        self.sibling_words = sibling_words
```

```

self.lp_words = lp_words
self.sector_group_dropout = sector_group_dropout # FIX 1
self.is_train = is_train # only dropout during training

# Build sibling lookup: (CompanyId, AsOfDate) -> list of df positions
print('Building sibling lookup...')
self.sibling_lookup = {}
for (cid, date), group in df.groupby(['CompanyId', 'AsOfDate']):
    self.sibling_lookup[(cid, date)] = group.index.tolist()
print(f' Lookup built: {len(self.sibling_lookup):,} company-date groups')

def __len__(self): return len(self.indices)

def _build_text(self, idx):
    row = self.df.iloc[idx]
    seg_name = str(row['SegmentName']).strip()
    seg_desc = str(row['SegmentDescription']).strip()
    long_p = str(row['LongProfile']).strip()
    year = str(row['AsOfDate'][:4])
    mstar = str(row['MstarGlobal']).strip()
    is_imputed = bool(row['segment_desc_imputed'])

    # FIX 1: Token dropout -----
    # 25% of training steps -> "[]" "[]" (inference-like, no hint)
    # 75% of training steps -> real "[104]" "[10420]" tokens
    # Eval always uses real tokens for clean measurement.
    use_real = (not self.is_train) or (_random.random() >= self.sector_group_dropout)
    if use_real and mstar:
        sector_tok = f"[{mstar[:3]}]"
        group_tok = f"[{mstar[:5]}]"
    else:
        sector_tok = "[]"
        group_tok = "[]"
    # -----

    # FIX 2: Respect self.lp_words (was hardcoded 60) -----
    lp_short = ''
    if long_p and not is_imputed:
        lp_short = ' '.join(long_p.split()[:self.lp_words])
    # -----

    # Sibling context (imputed word-cap preserved, no SegmentName fallback)
    orig_idx = row['index']
    sib_indices = self.sibling_lookup.get((row['CompanyId'], row['AsOfDate']), [])
    sib_parts = []
    for sib_idx in sib_indices:
        if sib_idx == orig_idx:
            continue
        sib = self.df[self.df['index'] == sib_idx].iloc[0]
        rev_pct = round(float(sib['revenue_share']) * 100, 1)
        sib_desc = str(sib['SegmentDescription']).strip()
        sib_is_imputed = bool(sib['segment_desc_imputed'])
        max_sib_words = 15 if sib_is_imputed else self.sibling_words
        sib_short = ' '.join(sib_desc.split()[:max_sib_words])
        sib_parts.append(f"[SEG {rev_pct}%] {sib_short}")

    parts = [
        sector_tok, group_tok, f"[{year}]", "[PRIMARY]",
        seg_name, "[SEP]", seg_desc,
    ]
    if sib_parts:
        parts.append(' '.join(sib_parts))
    if lp_short:
        parts.append(f"[LP] {lp_short}")

    return ' '.join(parts)

def __getitem__(self, pos):
    idx = self.indices[pos]
    text = self._build_text(idx)

    enc = self.tokenizer(
        text, max_length=self.max_len,
        padding='max_length', truncation=True, return_tensors='pt',
    )
    row = self.df.iloc[idx]
    return {

```

```

        'input_ids'      : enc['input_ids'].squeeze(0),
        'attention_mask': enc['attention_mask'].squeeze(0),
        'numeric_feats' : torch.tensor(self.numeric_feats[pos], dtype=torch.float32),
        'label'         : torch.tensor(row['label'], dtype=torch.long),
        'sec_label'     : torch.tensor(row['sector_label'], dtype=torch.long),
        'grp_label'     : torch.tensor(row['group_label'], dtype=torch.long),
    }

train_ds = GECSDataset(
    t1, t1_train_idx, train_numeric, tokenizer,
    CFG['max_len'], CFG['sibling_words'], CFG['lp_words'],
    sector_group_dropout=CFG['sector_group_dropout'],
    is_train=True,
)
test_ds = GECSDataset(
    t1, t1_test_idx, test_numeric, tokenizer,
    CFG['max_len'], CFG['sibling_words'], CFG['lp_words'],
    sector_group_dropout=CFG['sector_group_dropout'],
    is_train=False, # no dropout at eval - real tokens always used
)

train_loader = DataLoader(train_ds, batch_size=CFG['batch_size_train'],
                           shuffle=True, num_workers=2, pin_memory=True)
test_loader = DataLoader(test_ds, batch_size=CFG['batch_size_eval'],
                           shuffle=False, num_workers=2, pin_memory=True)

# Sanity check
print(f'Train batches: {len(train_loader)} | Test batches: {len(test_loader)}')
print('Token-dropout check (3 samples - expect mix of "[" and "[10x]'):'')
for pos in range(3):
    print(f' {train_ds._build_text(pos)[:140]}')

```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits a
 WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_

config.json: 100% 664/664 [00:00<00:00, 79.0kB/s]

tokenizer_config.json: 100% 369/369 [00:00<00:00, 49.8kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 6.95MB/s]

Building sibling lookup...

Lookup built: 23,207 company-date groups

Building sibling lookup...

Lookup built: 23,207 company-date groups

Train batches: 1302 | Test batches: 168

Token-dropout check (3 samples - expect mix of "[" and "[10x]'):''):

```

[ ] [ ] [2024] [PRIMARY] frozen and vegetables [SEP] frozen and vegetables segment includes the green giant and le su
[205] [20525] [2024] [PRIMARY] meals [SEP] meals segment includes, among others, the ortega, maple grove farms, cre
[205] [20525] [2024] [PRIMARY] specialty [SEP] specialty segment includes, among others, the crisco, clabber girl,

```

— Verify input text being fed to model —————

print('=== SAMPLE MODEL INPUTS (with all prefixes) ===')

print()

Build text for 5 sample rows

sample_indices = [0, 1, 2, 3, 4]

for pos in sample_indices:

idx = train_ds.indices[pos]

text = train_ds._build_text(idx)

row = train_ds.df.iloc[idx]

print(f'Row {pos} | class={row["MstarGlobal"]} | imputed={row["segment_desc_imputed"]} | tokens=~{len(text.split')}

print(f' {text}')

print()

Show one with siblings and one without

print('=== WITH SIBLINGS ===')

for pos in range(len(train_ds)):

idx = train_ds.indices[pos]

text = train_ds._build_text(idx)

if '[SEG]' in text:

row = train_ds.df.iloc[idx]

print(f'class={row["MstarGlobal"]} | n_segs={int(row["n_segments"])} | tokens=~{len(text.split())}')

print(f' {text}')

break

```

print()
print('=== WITHOUT SIBLINGS (single segment company) ===')
for pos in range(len(train_ds)):
    idx = train_ds.indices[pos]
    text = train_ds._build_text(idx)
    if 'SEG' not in text:
        row = train_ds.df.iloc[idx]
        print(f'class={row["MstarGlobal"]} | n_segs={int(row["n_segments"])} | tokens=~{len(text.split())}')
        print(f' {text}')
        break

print()
print('=== TOKEN LENGTH DISTRIBUTION (sample 500) ===')
lengths = []
for pos in range(min(500, len(train_ds))):
    idx = train_ds.indices[pos]
    text = train_ds._build_text(idx)
    lengths.append(len(text.split()))

lengths = sorted(lengths)
n = len(lengths)
print(f' median : {lengths[n//2]:.0f} tokens')
print(f' p90   : {lengths[int(n*0.9)]:.0f} tokens')
print(f' p99   : {lengths[int(n*0.99)]:.0f} tokens')
print(f' max    : {lengths[-1]:.0f} tokens')
print(f' >512   : {sum(1 for l in lengths if l > 512)}')

```

=== SAMPLE MODEL INPUTS (with all prefixes) ===

```

Row 0 | class=20525040 | imputed=False | tokens=~197
      [205] [20525] [2024] [PRIMARY] frozen and vegetables [SEP] frozen and vegetables segment includes the green giant a

Row 1 | class=20525040 | imputed=False | tokens=~196
      [205] [20525] [2024] [PRIMARY] meals [SEP] meals segment includes, among others, the ortega, maple grove farms, cre

Row 2 | class=20525040 | imputed=False | tokens=~207
      [205] [20525] [2024] [PRIMARY] specialty [SEP] specialty segment includes, among others, the crisco, clabber girl,

Row 3 | class=20525040 | imputed=False | tokens=~198
      [205] [20525] [2024] [PRIMARY] spices and flavor solutions [SEP] spices and flavor solutions segment includes, amon

Row 4 | class=31130010 | imputed=True | tokens=~144
      [311] [31130] [2024] [PRIMARY] raw materials and other [SEP] raw materials and other the company is a developer and

```

=== WITH SIBLINGS ===

```

class=20525040 | n_segs=4 | tokens=~197
[205] [20525] [2024] [PRIMARY] frozen and vegetables [SEP] frozen and vegetables segment includes the green giant a

```

=== WITHOUT SIBLINGS (single segment company) ===

```

class=10320020 | n_segs=1 | tokens=~118
[103] [10320] [2024] [PRIMARY] single segment [SEP] the company is a financial institution holding company. it is e

```

=== TOKEN LENGTH DISTRIBUTION (sample 500) ===

```

median : 177 tokens
p90    : 269 tokens
p99    : 389 tokens
max     : 393 tokens
>512   : 0

```

```

# — Model —————
class FLANGMultiTask(nn.Module):
    def __init__(self, model_name, n_leaf, n_sector, n_group, num_features, dropout=0.1):
        super().__init__()
        self.bert = AutoModel.from_pretrained(model_name)
        h = self.bert.config.hidden_size
        self.dropout = nn.Dropout(dropout)
        self.leaf_head = nn.Linear(h + num_features, n_leaf)
        self.sector_head = nn.Linear(h, n_sector)
        self.group_head = nn.Linear(h, n_group)

    def forward(self, input_ids, attention_mask, numeric_feats):
        out = self.bert(input_ids=input_ids, attention_mask=attention_mask)
        mask = attention_mask.unsqueeze(-1).float()
        pooled = (out.last_hidden_state * mask).sum(1) / mask.sum(1)
        pooled = self.dropout(pooled)
        combined = torch.cat([pooled, numeric_feats.to(pooled.dtype)], dim=-1)
        return (
            self.leaf_head(combined),
            self.sector_head(pooled),

```

```

        self.group_head(pooled),
    )

model = FLANGMultiTask(
    CFG['model_name'], NUM_CLASSES, NUM_SECTORS, NUM_GROUPS,
    CFG['num_features'], CFG['dropout']
).to(device)

total = sum(p.numel() for p in model.parameters())
print(f'Model      : {CFG["model_name"]}')
print(f'Parameters : {total/1e6:.1f}M')
print(f'Leaf head  : Linear({model.bert.config.hidden_size}+{CFG["num_features"]}, {NUM_CLASSES})')

```

pytorch_model.bin: 100% 438M/438M [00:04<00:00, 129MB/s]

Loading weights: 100% 197/197 [00:00<00:00, 8904.91it/s]

[transformers] BertModel LOAD REPORT from: SALT-NLP/FLANG-BERT

Key	Status
cls.predictions.transform.dense.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.predictions.decoder.bias	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.decoder.weight	UNEXPECTED
cls.predictions.bias	UNEXPECTED
pooler.dense.bias	MISSING
pooler.dense.weight	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstr

Model : SALT-NLP/FLANG-BERT

Parameters : 109.6M

Leaf head : Linear(768+10, 145)

— Loss & Metrics —————

```

class FocalLoss(nn.Module):
    def __init__(self, gamma=1.0, weight=None):
        super().__init__()
        self.gamma = gamma
        self.weight = weight

    def forward(self, logits, targets):
        ce = F.cross_entropy(logits, targets, weight=self.weight, reduction='none')
        pt = torch.exp(-ce)
        return ((1 - pt) ** self.gamma * ce).mean()

ce_leaf = nn.CrossEntropyLoss(weight=class_weights_t, label_smoothing=CFG['label_smoothing'])
ce_sector = nn.CrossEntropyLoss(weight=sector_weights_t, label_smoothing=CFG['label_smoothing'])
ce_group = nn.CrossEntropyLoss(weight=group_weights_t, label_smoothing=CFG['label_smoothing'])
focal_leaf = FocalLoss(gamma=CFG['focal_gamma'], weight=class_weights_t)

def compute_metrics(y_true, y_pred, y_probs=None):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    y_true_str = le.inverse_transform(y_true)
    macro_f1 = float(f1_score(y_true, y_pred, average='macro', zero_division=0))
    accuracy = float(accuracy_score(y_true, y_pred))
    top10_f1 = float(f1_score(y_true, y_pred, labels=t1_top_k_enc, average='macro', zero_division=0))
    sectors_true = pd.Series(y_true_str).map(industry_to_sector).values
    sf1s = []
    for sec in pd.Series(sectors_true).dropna().unique():
        mask = sectors_true == sec
        if mask.sum() < 5: continue
        sc = [ind for ind, s in industry_to_sector.items() if s == sec and ind in le.classes_]
        se = le.transform(sc)
        sf1s.append(float(f1_score(y_true[mask], y_pred[mask], labels=se, average='macro', zero_division=0)))
    within_sector_f1 = float(np.mean(sf1s)) if sf1s else 0.0
    coverage_70 = float((y_probs.max(axis=1) > 0.7).mean()) if y_probs is not None else 0.0
    return {'macro_f1': macro_f1, 'top10_f1': top10_f1,
            'within_sector_f1': within_sector_f1, 'coverage_70': coverage_70, 'accuracy': accuracy}

print('Losses and metrics ready.')

```

Losses and metrics ready.

```
# — LLRD Optimizer —————
def get_llrd_optimizer(model, base_lr, decay=0.95):
    params = []
    params.append({'params': list(model.leaf_head.parameters()) +
                        list(model.sector_head.parameters()) +
                        list(model.group_head.parameters()), 'lr': base_lr})
    if hasattr(model.bert, 'encoder'):
        layers = model.bert.encoder.layer
        for i, layer in enumerate(reversed(layers)):
            params.append({'params': layer.parameters(), 'lr': base_lr * (decay ** (i+1))})
            params.append({'params': model.bert.embeddings.parameters(),
                            'lr': base_lr * (decay ** (len(layers)+1))})
    else:
        params.append({'params': model.bert.parameters(), 'lr': base_lr * 0.5})
    return torch.optim.AdamW(params, weight_decay=0.01)

print(f'LLRD optimizer ready - decay={CFG["llrd_factor"]}')

```

LLRD optimizer ready - decay=0.95

```
# — Train / Eval Functions —————
def train_epoch(model, loader, optimizer, scheduler, leaf_loss_fn):
    model.train()
    total_loss, n = 0.0, 0
    for batch in loader:
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        numeric_feats = batch['numeric_feats'].to(device)
        labels = batch['label'].to(device)
        sec_labels = batch['sec_label'].to(device)
        grp_labels = batch['grp_label'].to(device)
        optimizer.zero_grad()
        with torch.autocast(device_type='cuda', dtype=torch.bfloat16):
            leaf_logits, sec_logits, grp_logits = model(input_ids, attention_mask, numeric_feats)
            loss = (
                leaf_loss_fn(leaf_logits, labels)
                + CFG['aux_weight_sector'] * ce_sector(sec_logits, sec_labels)
                + CFG['aux_weight_group'] * ce_group(grp_logits, grp_labels)
            )
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()
            scheduler.step()
            total_loss += loss.item() * len(labels)
            n += len(labels)
    return total_loss / n

@torch.no_grad()
def evaluate(model, loader):
    model.eval()
    all_preds, all_labels, all_probs = [], [], []
    for batch in loader:
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        numeric_feats = batch['numeric_feats'].to(device)
        labels = batch['label'].to(device)
        with torch.autocast(device_type='cuda', dtype=torch.bfloat16):
            leaf_logits, _, _ = model(input_ids, attention_mask, numeric_feats)
        probs = torch.softmax(leaf_logits.float(), dim=-1)
        preds = probs.argmax(dim=-1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())
        all_probs.extend(probs.cpu().numpy())
    y_true = np.array(all_labels)
    y_pred = np.array(all_preds)
    y_probs = np.array(all_probs)
    return compute_metrics(y_true, y_pred, y_probs), y_pred, y_probs

def print_row(phase, ep, total_ep, loss, m, lr, elapsed):
    print(
        f' Ph{phase} Ep{ep}/{total_ep}'
        f' loss={loss:.4f}'
        f' macro={m["macro_f1"]:.4f}'
        f' w-sec={m["within_sector_f1"]:.4f}'
        f' top10={m["top10_f1"]:.4f}'
        f' cov70={m["coverage_70"]:.3f}'
    )

```



```

        f' lr={lr:.2e}'
        f' [{elapsed:.0f}s]'
    )

print('Train/eval functions ready.')

```

Train/eval functions ready.

```

# — 3-Phase Training —————
def run_phase(phase_num, epochs, lr, leaf_loss_fn, model, best_macro, best_state, history, use_llrd=True):
    optimizer = get_llrd_optimizer(model, lr, CFG['llrd_factor']) if use_llrd else torch.optim
    total_steps = len(train_loader) * epochs
    warmup_steps = int(total_steps * CFG['warmup_ratio'])
    scheduler = get_cosine_schedule_with_warmup(optimizer, warmup_steps, total_steps)
    phase_best = 0.0
    phase_best_ep = 0
    no_improve = 0
    PATIENCE = 3
    loss_name = leaf_loss_fn.__class__.__name__
    print(f'{"-"*75}')
    print(f'PHASE {phase_num} | epochs={epochs} lr={lr} loss={loss_name} LLRD={use_llrd}')
    print(f' Ph Ep loss macro w-sec top10 cov70 lr time')
    print(f'{"-"*75}')
    for ep in range(1, epochs + 1):
        t0 = time.time()
        train_loss = train_epoch(model, train_loader, optimizer, scheduler, leaf_loss_fn)
        m, _, _ = evaluate(model, test_loader)
        elapsed = time.time() - t0
        current_lr = scheduler.get_last_lr()[0]
        print_row(phase_num, ep, epochs, train_loss, m, current_lr, elapsed)
        history.append({
            'phase': phase_num, 'epoch': ep,
            'train_loss': round(train_loss, 4),
            'macro_f1': round(m['macro_f1'], 4),
            'within_sector_f1': round(m['within_sector_f1'], 4),
            'top10_f1': round(m['top10_f1'], 4),
            'coverage_70': round(m['coverage_70'], 4),
            'accuracy': round(m['accuracy'], 4),
            'lr': current_lr, 'loss_fn': loss_name,
        })
        if m['macro_f1'] > best_macro:
            best_macro = m['macro_f1']
            phase_best = m['macro_f1']
            phase_best_ep = ep
            no_improve = 0
            best_state = {k: v.cpu().clone() for k, v in model.state_dict().items()}
            print(f' NEW BEST macro={best_macro:.4f} (Phase {phase_num} Epoch {ep})')
        else:
            no_improve += 1
            print(f' no improvement {no_improve}/{PATIENCE}')
            if no_improve >= PATIENCE:
                print(f' Early stopping – best was epoch {phase_best_ep}')
                break
    print(f'Phase {phase_num} done – best: {phase_best:.4f} at epoch {phase_best_ep}')
    return best_macro, best_state, history

best_macro = 0.0
best_state = None
history = []

# Phase 1 – CE warmup
best_macro, best_state, history = run_phase(
    1, CFG['phase1_epochs'], CFG['lr_phase1'], ce_leaf,
    model, best_macro, best_state, history, use_llrd=True
)
torch.save(best_state, ART_DIR / 'best_model_ph1.pt')
print(f'Phase 1 saved – macro={best_macro:.4f}')

# Phase 2 – Focal
best_macro, best_state, history = run_phase(
    2, CFG['phase2_epochs'], CFG['lr_phase2'], focal_leaf,
    model, best_macro, best_state, history, use_llrd=True
)
torch.save(best_state, ART_DIR / 'best_model_ph2.pt')
print(f'Phase 2 saved – macro={best_macro:.4f}')

# Phase 3 – CE low LR

```

```

best_macro, best_state, history = run_phase(
    3, CFG['phase3_epochs'], CFG['lr_phase3'], ce_leaf,
    model, best_macro, best_state, history, use_llrd=False
)

print(f'{" "*75}')
print(f'TRAINING COMPLETE - Best macro: {best_macro:.4f}')
best_row = max(history, key=lambda x: x['macro_f1'])
print(f'Best phase: {best_row["phase"]} Best epoch: {best_row["epoch"]}')
print(f'{" "*75}')

```

```

=====
PHASE 1 | epochs=6 lr=2e-05 loss=CrossEntropyLoss LLRD=True
Ph Ep loss macro w-sec top10 cov70 lr time
=====
model.safetensors: 100% 438M/438M [00:04<00:00, 151MB/s]
Ph1 Ep1/6 loss=4.0849 macro=0.6568 w-sec=0.6482 top10=0.8866 cov70=0.331 lr=1.97e-05 [174s]
NEW BEST macro=0.6568 (Phase 1 Epoch 1)
Ph1 Ep2/6 loss=1.7656 macro=0.7752 w-sec=0.7579 top10=0.9164 cov70=0.689 lr=1.69e-05 [172s]
NEW BEST macro=0.7752 (Phase 1 Epoch 2)
Ph1 Ep3/6 loss=1.4592 macro=0.8048 w-sec=0.7864 top10=0.9253 cov70=0.792 lr=1.17e-05 [172s]
NEW BEST macro=0.8048 (Phase 1 Epoch 3)
Ph1 Ep4/6 loss=1.3374 macro=0.8138 w-sec=0.7994 top10=0.9261 cov70=0.829 lr=6.04e-06 [173s]
NEW BEST macro=0.8138 (Phase 1 Epoch 4)
Ph1 Ep5/6 loss=1.2689 macro=0.8253 w-sec=0.8119 top10=0.9278 cov70=0.839 lr=1.65e-06 [172s]
NEW BEST macro=0.8253 (Phase 1 Epoch 5)
Ph1 Ep6/6 loss=1.2310 macro=0.8263 w-sec=0.8136 top10=0.9261 cov70=0.844 lr=0.00e+00 [173s]
NEW BEST macro=0.8263 (Phase 1 Epoch 6)
Phase 1 done - best: 0.8263 at epoch 6
Phase 1 saved - macro=0.8263
=====
PHASE 2 | epochs=4 lr=2e-05 loss=FocalLoss LLRD=True
Ph Ep loss macro w-sec top10 cov70 lr time
=====
Ph2 Ep1/4 loss=0.4401 macro=0.8058 w-sec=0.8010 top10=0.9156 cov70=0.779 lr=1.87e-05 [172s]
no improvement 1/3
Ph2 Ep2/4 loss=0.4088 macro=0.8266 w-sec=0.8170 top10=0.9182 cov70=0.790 lr=1.17e-05 [172s]
NEW BEST macro=0.8266 (Phase 2 Epoch 2)
Ph2 Ep3/4 loss=0.3531 macro=0.8246 w-sec=0.8193 top10=0.9279 cov70=0.822 lr=3.57e-06 [172s]
no improvement 1/3
Ph2 Ep4/4 loss=0.3269 macro=0.8267 w-sec=0.8229 top10=0.9227 cov70=0.822 lr=0.00e+00 [172s]
NEW BEST macro=0.8267 (Phase 2 Epoch 4)
Phase 2 done - best: 0.8267 at epoch 4
Phase 2 saved - macro=0.8267
=====
PHASE 3 | epochs=4 lr=5e-06 loss=CrossEntropyLoss LLRD=False
Ph Ep loss macro w-sec top10 cov70 lr time
=====
Ph3 Ep1/4 loss=1.1419 macro=0.8229 w-sec=0.8142 top10=0.9296 cov70=0.868 lr=4.66e-06 [172s]
no improvement 1/3
Ph3 Ep2/4 loss=1.1023 macro=0.8260 w-sec=0.8192 top10=0.9288 cov70=0.882 lr=2.93e-06 [172s]
no improvement 2/3
Ph3 Ep3/4 loss=1.0715 macro=0.8279 w-sec=0.8214 top10=0.9295 cov70=0.882 lr=8.93e-07 [172s]
NEW BEST macro=0.8279 (Phase 3 Epoch 3)
Ph3 Ep4/4 loss=1.0604 macro=0.8288 w-sec=0.8224 top10=0.9296 cov70=0.884 lr=0.00e+00 [172s]
NEW BEST macro=0.8288 (Phase 3 Epoch 4)
Phase 3 done - best: 0.8288 at epoch 4
=====
TRAINING COMPLETE - Best macro: 0.8288
Best phase: 3 Best epoch: 4
=====

```

```

# — Final Evaluation —————
model.load_state_dict(best_state)
final_m, y_pred_final, y_probs_final = evaluate(model, test_loader)
y_true_final = t1.iloc[t1_test_idx]['label'].values

print()
print('-----+')
print('| TASK 1 - FLANG-BERT v10 FINAL RESULTS |')
print('-----+')
print(f'| macro F1 : {final_m["macro_f1"]:.4f} {"ABOVE" if final_m["macro_f1"]>=0.75 else "below"} bar (0)')
print(f'| top-10 F1 : {final_m["top10_f1"]:.4f} {"ABOVE" if final_m["top10_f1"]>=0.85 else "below"} bar (0)')
print(f'| within-sector F1 : {final_m["within_sector_f1"]:.4f} |')
print(f'| coverage @ 0.7 : {final_m["coverage_70"]:.4f} |')
print(f'| accuracy : {final_m["accuracy"]:.4f} |')
print('-----+')

y_true_str = le.inverse_transform(y_true_final)
sectors = pd.Series(y_true_str).map(industry_to_sector).values

```

```

print('-- Per-Sector F1 --')
for sec in sorted(pd.Series(sectors).dropna().unique()):
    mask = sectors == sec
    sc = [ind for ind, s in industry_to_sector.items() if s == sec and ind in le.classes_]
    se = le.transform(sc)
    sf1 = float(f1_score(y_true_final[mask], y_pred_final[mask], labels=se, average='macro', zero_division=0))
    flag = 'OK' if sf1 >= 0.75 else ' '
    print(f' {flag} {sec:30s} {sf1:.3f} n={mask.sum():,}')

```

```

+-----+
| TASK 1 - FLANG-BERT v10 FINAL RESULTS |
+-----+
| macro F1      : 0.8288  ABOVE bar (0.75) |
| top-10 F1     : 0.9296  ABOVE bar (0.85) |
| within-sector F1 : 0.8224 |
| coverage @ 0.7  : 0.8836 |
| accuracy       : 0.8784 |
+-----+

```

```

-- Per-Sector F1 --
OK Basic Materials          0.834 n=881
OK Communication Services   0.891 n=529
OK Consumer Cyclical        0.892 n=1,195
OK Consumer Defensive        0.889 n=700
OK Energy                   0.802 n=344
OK Financial Services        0.795 n=1,568
OK Healthcare                0.744 n=902
OK Industrials               0.858 n=2,253
OK Real Estate               0.787 n=780
OK Technology                0.754 n=1,202
OK Utilities                 0.801 n=369

```

```

# -- Save All Artifacts --
torch.save(best_state, ART_DIR / 'best_model_state.pt')
np.save(ART_DIR / 'task1_flangbert_v10_test_logits.npy', y_probs_final)

with open(ART_DIR / 'label_encoder.pkl', 'wb') as f:
    pickle.dump(le, f)
with open(ART_DIR / 'label_encoder_sector.pkl', 'wb') as f:
    pickle.dump(le_sec, f)
with open(ART_DIR / 'label_encoder_group.pkl', 'wb') as f:
    pickle.dump(le_grp, f)

pd.DataFrame({
    'CompanyId' : t1.iloc[t1_test_idx]['CompanyId'].values,
    'SegmentName': t1.iloc[t1_test_idx]['SegmentName'].values,
    'y_true'    : le.inverse_transform(y_true_final),
    'y_pred'    : le.inverse_transform(y_pred_final),
    'correct'   : y_true_final == y_pred_final,
    'confidence' : y_probs_final.max(axis=1).round(4),
}).to_csv(ART_DIR / 'task1_flangbert_v10_predictions.csv', index=False)

pd.DataFrame(history).to_csv(ART_DIR / 'training_history.csv', index=False)

json.dump({
    'timestamp'      : datetime.now().isoformat(timespec='seconds'),
    'run_name'       : 'task1_flangbert_v10_clean_arch',
    'macro_f1'       : final_m['macro_f1'],
    'top10_f1'       : final_m['top10_f1'],
    'within_sector_f1' : final_m['within_sector_f1'],
    'coverage_70'     : final_m['coverage_70'],
    'accuracy'       : final_m['accuracy'],
    'best_phase'     : best_row['phase'],
    'best_epoch'     : best_row['epoch'],
    'config'         : CFG,
}, open(ART_DIR / 'results_summary.json', 'w'), indent=2)

print('All artifacts saved:')
print(' best_model_state.pt')
print(' task1_flangbert_v10_test_logits.npy')
print(' label_encoder.pkl')
print(' numeric_scaler.pkl')
print(' task1_flangbert_v10_predictions.csv')
print('Final macro F1: ' + str(round(final_m['macro_f1'], 4)))

```

```

All artifacts saved:
best_model_state.pt
task1_flangbert_v10_test_logits.npy
label_encoder.pkl

```

```
numeric_scaler.pkl  
task1_flangbert_v10_predictions.csv  
Final macro F1: 0.8288
```